```python
import heapq

# 0 represents the blank space
initial_state = (
    2, 8, 3,
    1, 6, 4,
    7, 0, 5
)

goal_state = (
    1, 2, 3,
    8, 0, 4,
    7, 6, 5
)


# Calculate Manhattan distance
def heuristic(state):
    distance = 0

    for i in range(9):
        tile = state[i]

        if tile != 0:
            current_row, current_col = divmod(i, 3)
            goal_index = goal_state.index(tile)
            goal_row, goal_col = divmod(goal_index, 3)

            distance += abs(current_row - goal_row)
            distance += abs(current_col - goal_col)

    return distance


# Generate neighboring states
def get_neighbors(state):
    neighbors = []

    blank = state.index(0)
    row, col = divmod(blank, 3)

    moves = [
        (-1, 0),  # Up
        (1, 0),   # Down
        (0, -1),  # Left
        (0, 1)    # Right
    ]

    for dr, dc in moves:
```

```
            new_row = row + dr
            new_col = col + dc

            if 0 <= new_row < 3 and 0 <= new_col < 3:
                new_blank = new_row * 3 + new_col

                new_state = list(state)
                new_state[blank], new_state[new_blank] = (
                    new_state[new_blank], new_state[blank]
                )

                neighbors.append(tuple(new_state))

        return neighbors


# A* search algorithm
def a_star(start, goal):
    # Priority queue stores (f, g, state)
    open_list = [(heuristic(start), 0, start)]

    came_from = {}
    g_score = {start: 0}
    closed_set = set()

    while open_list:
        f, g, current = heapq.heappop(open_list)

        if current in closed_set:
            continue

        closed_set.add(current)

        if current == goal:
            path = [current]

            while current in came_from:
                current = came_from[current]
                path.append(current)

            return path[::-1]

        for neighbor in get_neighbors(current):
            tentative_g = g + 1

            if tentative_g < g_score.get(
                neighbor, float("inf")
            ):
                g_score[neighbor] = tentative_g
```

```
                came_from[neighbor] = current

                h = heuristic(neighbor)
                f = tentative_g + h

                heapq.heappush(
                    open_list, (f, tentative_g, neighbor)
                )

    return None

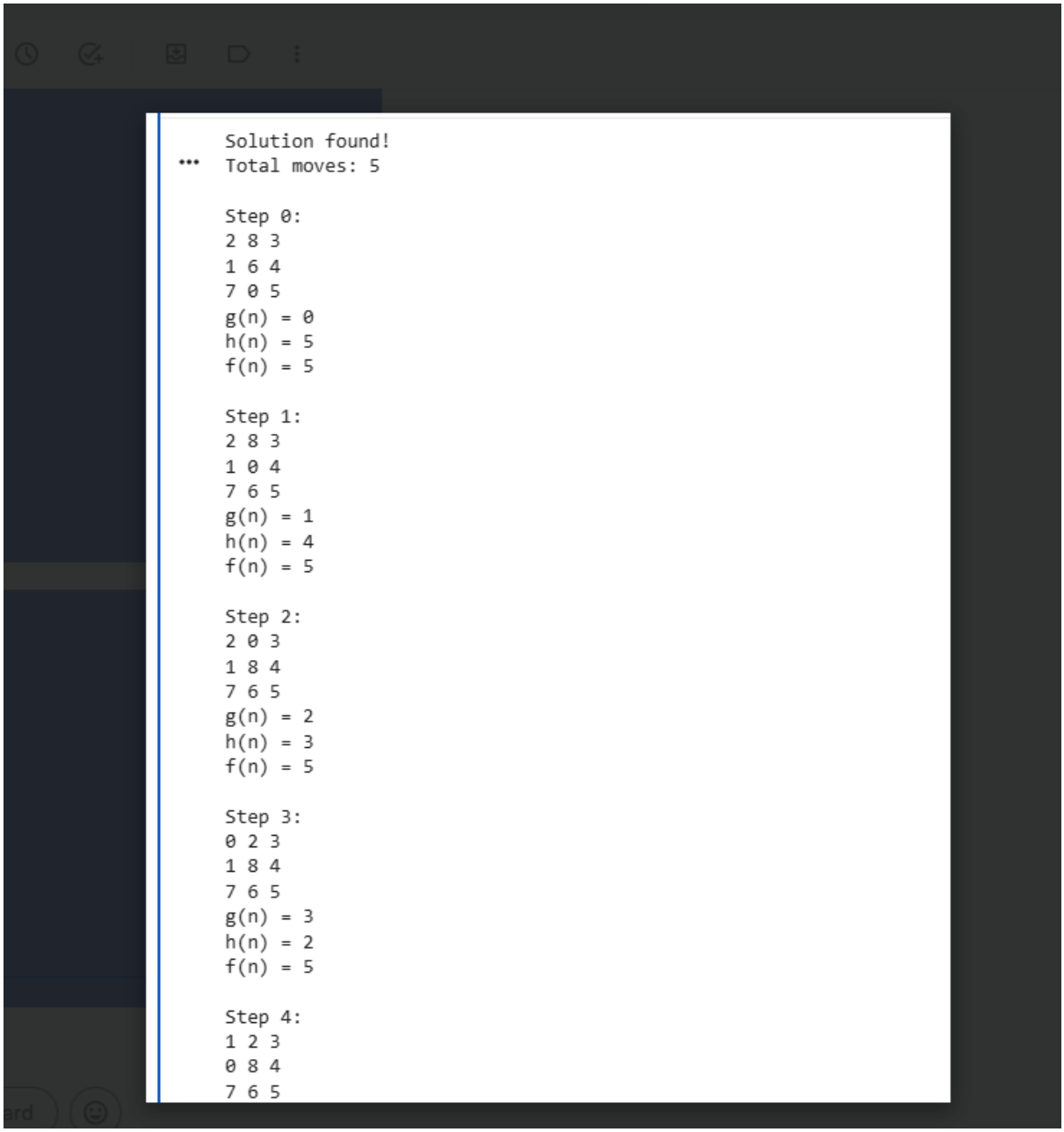

# Display the solution
path = a_star(initial_state, goal_state)

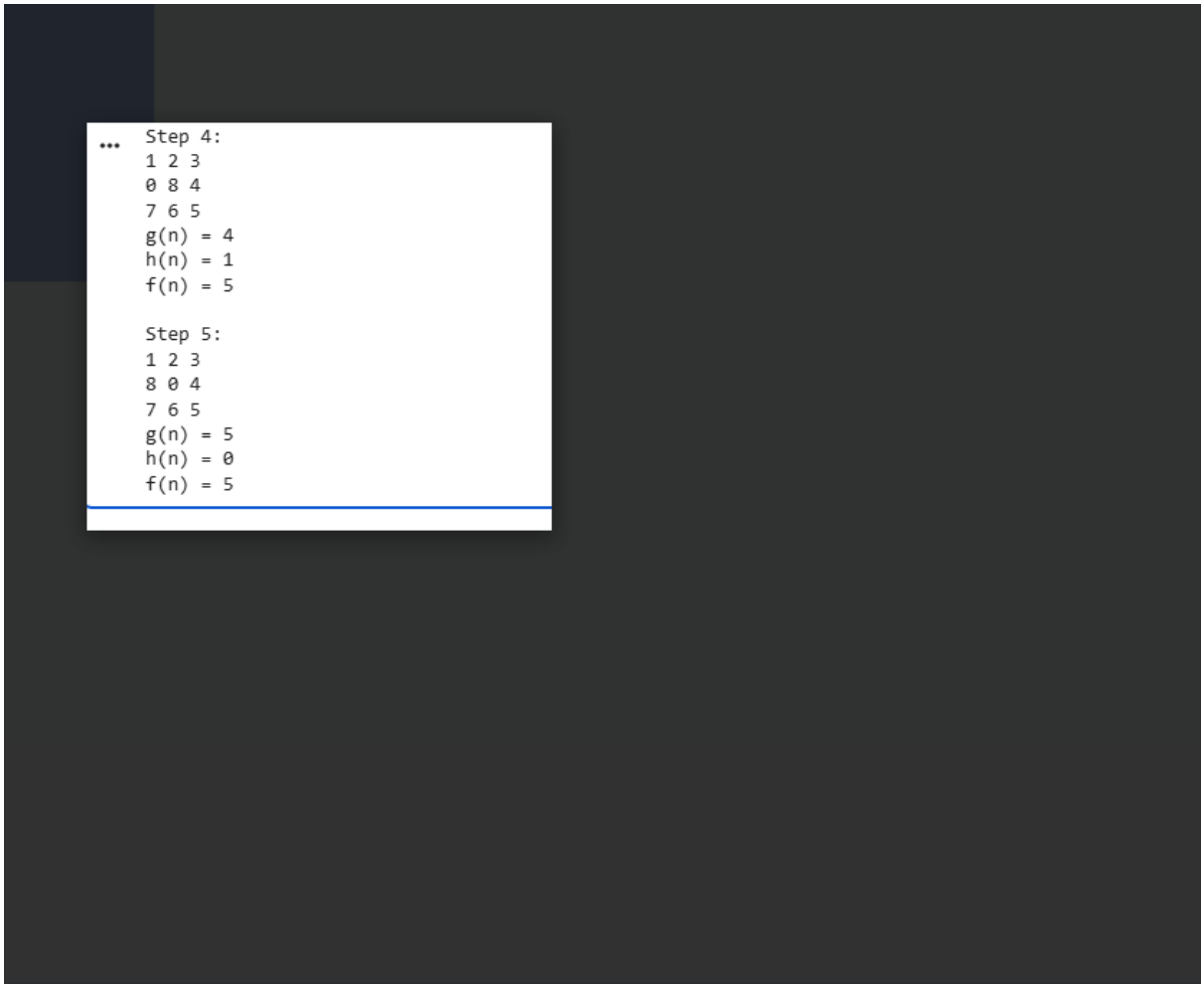
if path:
    print("Solution found!")
    print("Total moves:", len(path) - 1)

    for step, state in enumerate(path):
        print(f"\nStep {step}:")
        for i in range(0, 9, 3):
            print(*state[i:i + 3])

        print("g(n) =", step)
        print("h(n) =", heuristic(state))
        print("f(n) =", step + heuristic(state))
else:
    print("No solution exists.")
```

Output


Solution found!
Total moves: 5
Step 0:
2 8 3
1 6 4
7 0 5
g(n) = 0
h(n) = 5
f(n) = 5
Step 1:
2 8 3
1 0 4
7 6 5
g(n) = 1
h(n) = 4
f(n) = 5
Step 2:
2 0 3
1 8 4
7 6 5
g(n) = 2
h(n) = 3
f(n) = 5
Step 3:
0 2 3
1 8 4
7 6 5
g(n) = 3
h(n) = 2
f(n) = 5
Step 4:
1 2 3
0 8 4
7 6 5



A*Manhattan

```
import heapq

# 0 represents the blank space
initial_state = (
    2, 8, 3,
    1, 6, 4,
    7, 0, 5
)

goal_state = (
    1, 2, 3,
    8, 0, 4,
    7, 6, 5
)
```

```
# Calculate Manhattan distance
def heuristic(state):
    distance = 0

    for i in range(9):
        tile = state[i]

        if tile != 0:
            current_row, current_col = divmod(i, 3)
            goal_index = goal_state.index(tile)
            goal_row, goal_col = divmod(goal_index, 3)

            distance += abs(current_row - goal_row)
            distance += abs(current_col - goal_col)

    return distance


# Generate neighboring states
def get_neighbors(state):
    neighbors = []

    blank = state.index(0)
    row, col = divmod(blank, 3)

    moves = [
        (-1, 0),  # Up
        (1, 0),   # Down
        (0, -1),  # Left
        (0, 1)    # Right
    ]

    for dr, dc in moves:
        new_row = row + dr
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_blank = new_row * 3 + new_col

            new_state = list(state)
            new_state[blank], new_state[new_blank] = (
                new_state[new_blank], new_state[blank]
            )

            neighbors.append(tuple(new_state))

    return neighbors


# A* search algorithm
```

```
def a_star(start, goal):
    # Priority queue stores (f, g, state)
    open_list = [(heuristic(start), 0, start)]

    came_from = {}
    g_score = {start: 0}
    closed_set = set()

    while open_list:
        f, g, current = heapq.heappop(open_list)

        if current in closed_set:
            continue

        closed_set.add(current)

        if current == goal:
            path = [current]

            while current in came_from:
                current = came_from[current]
                path.append(current)

            return path[::-1]

        for neighbor in get_neighbors(current):
            tentative_g = g + 1

            if tentative_g < g_score.get(
                neighbor, float("inf")
            ):
                g_score[neighbor] = tentative_g
                came_from[neighbor] = current

                h = heuristic(neighbor)
                f = tentative_g + h

                heapq.heappush(
                    open_list, (f, tentative_g, neighbor)
                )

    return None

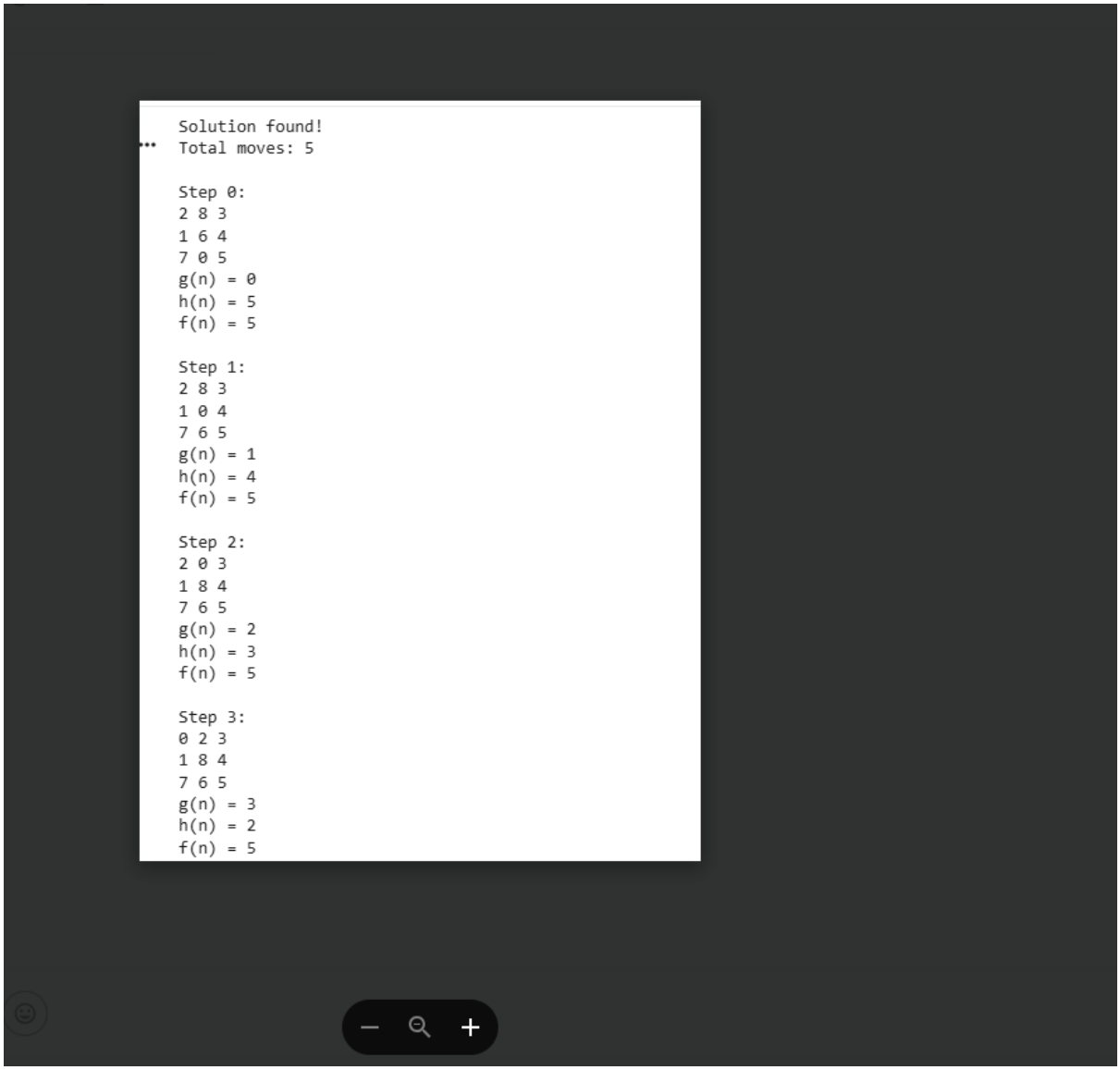

# Display the solution
path = a_star(initial_state, goal_state)

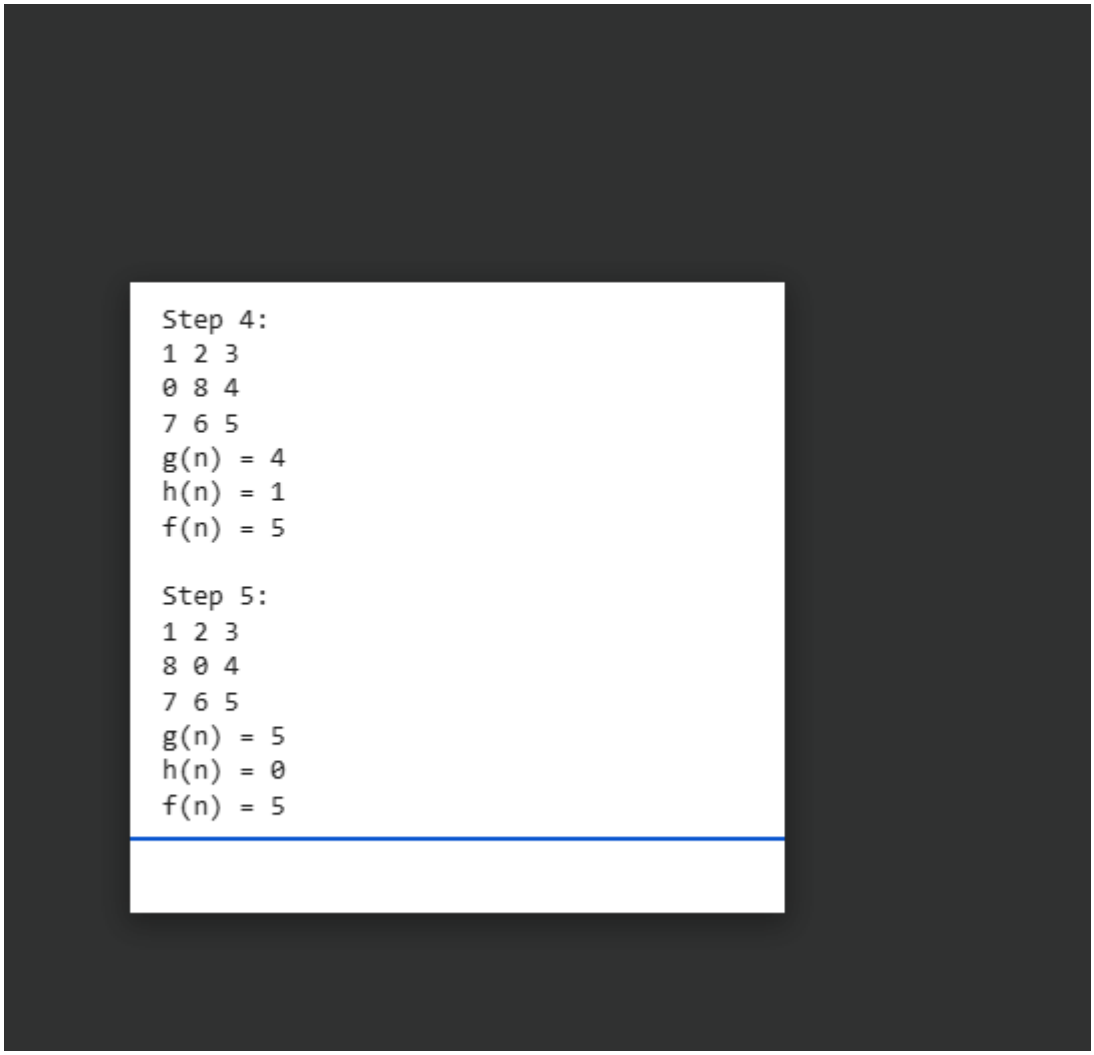
if path:
    print("Solution found!")
    print("Total moves:", len(path) - 1)
```

```
    for step, state in enumerate(path):
        print(f"\nStep {step}:")
        for i in range(0, 9, 3):
            print(*state[i:i + 3])

        print("g(n) =", step)
        print("h(n) =", heuristic(state))
        print("f(n) =", step + heuristic(state))
else:
    print("No solution exists.")
```

output:




Step 4:
1 2 3
0 8 4
7 6 5
g(n) = 4
h(n) = 1
f(n) = 5
Step 5:
1 2 3
8 0 4
7 6 5
g(n) = 5
h(n) = 0
f(n) = 5